ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

الجـــمهوريـــة الجزائريـــة الديمـــقراطيـــة الشـعبيـة
وزارة التعليـــم العـــالي والبـــحث العلمـــي
People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Intelligent and Communicating Systems, ICS

2^{nd} Year Specialty SIQ G02

Lab Report n°02

Title:

Arduino Communications GPIO-Sensors-Actuators

Studied by:

First Name: Abderrahmane

Last Name: GRINE

E_mail: ma_grine@esi.dz

A. Theory

1. Pushbutton (Digital Input)

A pushbutton is an electromechanical switch used to provide user input to a microcontroller. When pressed, it establishes an electrical connection between its terminals, which can be detected as a digital logic level by a GPIO pin.

Since a pushbutton is passive equipment, it must be connected in a way that ensures a valid *HIGH* or *LOW* logic level. An internal or external pull-(up/down) resistor to fix the default state of the input when the button is not pressed.

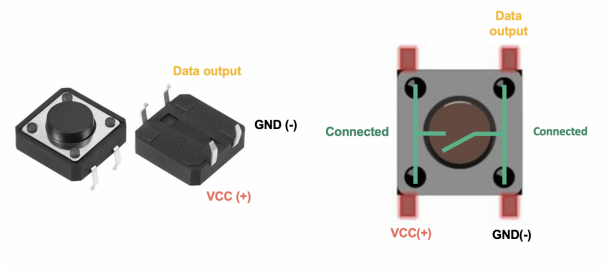

Figure 1: Pushbutton schematic

2. Digital GPIO Features of the MKR1010

- Logic voltage level: **3.3 V** (not 5 V tolerant)
- Number of digital I/O pins: 15 (D0 - D14)
- Maximum recommended current per pin: 7 mA
- Some pins support PWM, interrupts or serial communication

3. Analogue GPIO Features of the MKR1010

- Number of analogue inputs: 7 (A0–A6) with **12 bits** (0–4095) ADC resolution
- One true DAC output (A0) with **10-bit** (0–1023) resolution
- Input voltage range: 0–3.3 V
- Recommended source impedance for accurate ADC conversion: $\leq 10\,\text{k}\Omega$

4. Debounce Circuit: Is It Necessary?

A mechanical pushbutton does not produce a clean transition between ON and OFF states. Due to finger-induced contact bounce, several rapid transitions may occur within a few milliseconds, which can cause the microcontroller to detect multiple presses.

- **Software debouncing**: ignore rapid changes using a short delay (10–50 ms).
- **Hardware debouncing**: use an RC filter (e.g. 10 kΩ + 100 nF) or a Schmitt trigger (see figure 2).

Debouncing is therefore **necessary** in most applications to ensure a single, stable logic transition.

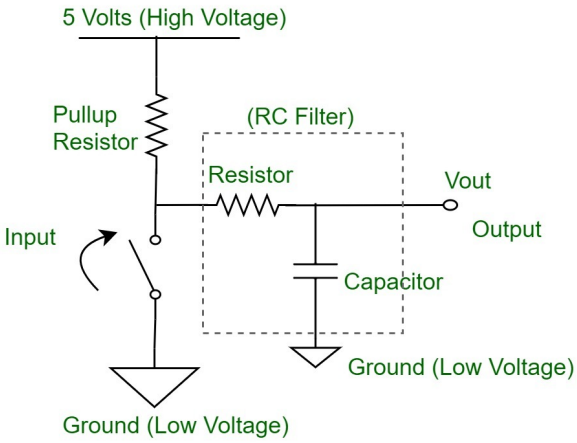

Figure 2: Debounce circuit

5. Pull-Up and Pull-Down Resistors

A GPIO input must not be left floating, otherwise it may randomly read HIGH or LOW due to noise.

- When a **pull-up resistor** is used, the input is normally at logic HIGH, and transitions to LOW when the button is pressed.
- When a **pull-down resistor** is used, the input is normally at logic LOW, and transitions to HIGH when the button is pressed.

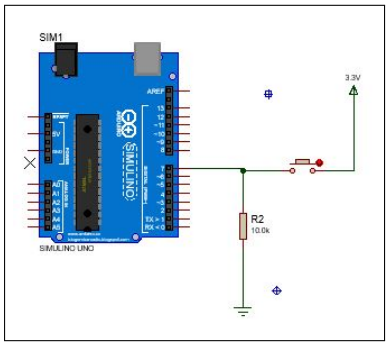

Figure 3: pull down resistor schema.

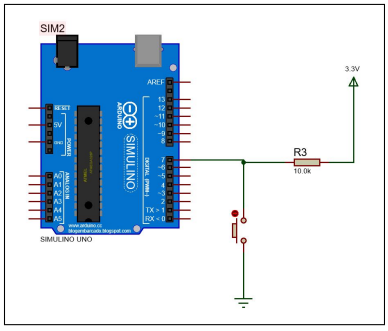

Figure 4: pull up resistor schema.

6. Connection of a Light Sensor (Analog GPIO)

6.1. Light Sensor Introduction

A light sensor is an electronic component that can detect the intensity of light and converts it's energy into an electrical signal. A Light-Dependent Resistor (LDR) changes its resistance based on light intensity. It is commonly used to detect ambient light in automation systems.

6.2. Using the Serial Monitor

The Arduino IDE Serial Monitor can display sensor values in real-time. Use `Serial.begin(9600);` to initialize communication and `Serial.println()` to print values.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println"(Hello world"!) ;
  delay (1000) ;
}
```

6.3. Analog GPIO Configuration

To configure a GPIO (General-Purpose Input/Output) line as an analog output on an Arduino, the `analogWrite()` function is used. This function outputs a PWM (Pulse Width Modulation) signal that simulates an analog signal.

II. ACTIVITY

1. Connection of a Light Sensor (Analog)

1.1. Hardware

The setup includes a Light Dependent Resistor (LDR) configured as part of a voltage divider circuit. One terminal of the LDR is connected to the 3.3V output of the Arduino, the other terminal is connected to analog input pin `A1` (ADC) and to a 10kΩ resistor that is further connected to the ground.

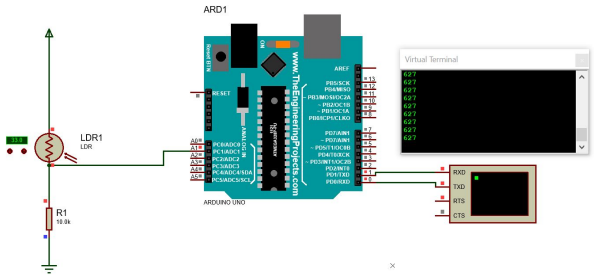

Figure 1.1: LDR display schematic

1.2. Software

The Arduino reads the output voltage from the LDR using the `analogRead()` function. The digital value obtained (12 bits ADC) is then printed to the serial monitor .

```
const int LDR_PIN = A1; // define the analog pin connected to the LDR
int lightValue = 0; // variable to store the light intensity
void setup() {
  Serial.begin(9600);  // Arduino serial console as a display.
}
void loop() {
  lightValue = analogRead(LDR_PIN);    // configuring a GPIO line as an
   analog input
  Serial.println(lightValue); // display the light value in a terminal.
  delay(500);
}
```

Listing 1.1: Reading LDR values using analog input

1.3. Implementation

After wiring the circuit and uploading the code, the Arduino serial monitor can be activated to display the ADC readings from the LDR. The closer the value is to 4095, the more intense the ambient light.

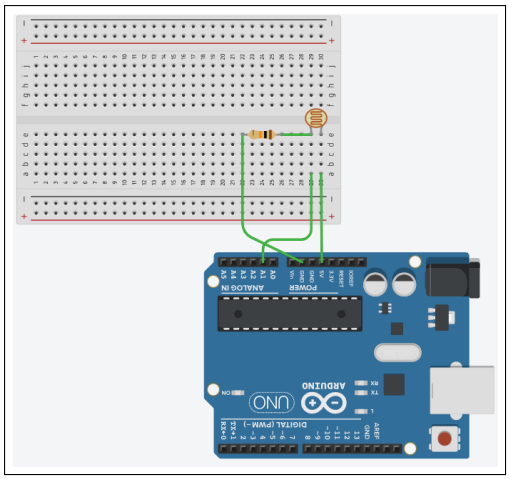

Figure 1.2: LDR circuit with Thinker cad.

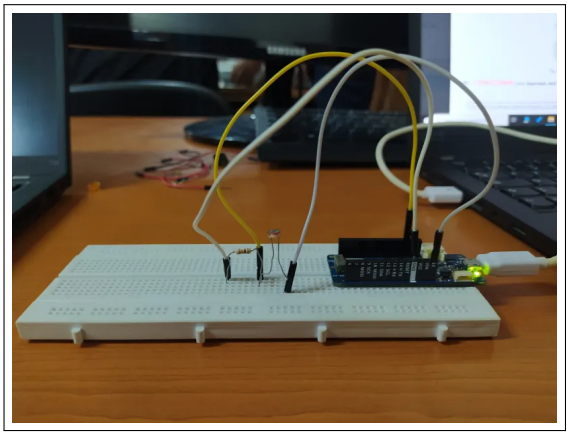

Figure 1.3: Simulation of the configuration.

2. Connection of a Push Button (Digital)

2.1. Hardware

A push button is connected to a digital GPIO pin. To avoid floating input, either a **pull-up** or **pull-down** resistor is used:

- **Pull-Up:** GPIO is connected to `Vcc` through a resistor (reads `HIGH` by default). When pressed, the pin is connected to `GND` (reads `LOW`). (see Figure 4)
- **Pull-Down:** GPIO is connected to `GND` through a resistor (reads `LOW` by default). When pressed, the pin is connected to `Vcc` (reads `HIGH`). (see Figure 3)

2.2. Software

Pull-down

```
const int BUTTON_PIN = 7;
// Variable to store the current button state
int buttonState = 0;
void setup() {
  // Configure BUTTON_PIN as a digital input
  pinMode(BUTTON_PIN, INPUT);    // --> Pin is connected to GND through the resistor (default: LOW)
  Serial.begin(9600);  // Initialize serial communication for debugging
}
void loop() {
  buttonState = digitalRead(BUTTON_PIN); // Read the button state (LOW or HIGH)
  // With PULL-DOWN:
  // --> LOW  = Button not pressed
  // --> HIGH = Button pressed

```

```
15    // If using PULL-UP:
16    // --> LOW  = Button pressed
17    // --> HIGH = Button not pressed
18    if (buttonState == HIGH)  // Button is pressed with pull-down
19      Serial.println("Button Pressed");
20    else
21      Serial.println("Button Released");
22    delay(100); // Delay to make the output readable
23  }
```

Listing 1.2: Pushbutton handling with pull-down resistor

2.3. Implementation

In the figure below, we show the behavior of the pushbutton circuit using both Proteus simulation (a) and a physical, real-life setup (b).

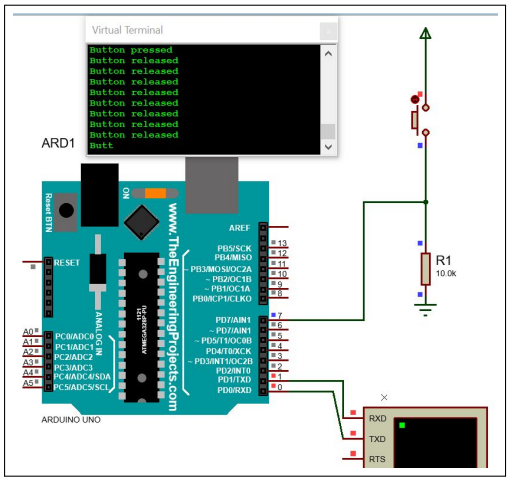

Figure 1.4: pushbutton state display circuit on proteus.

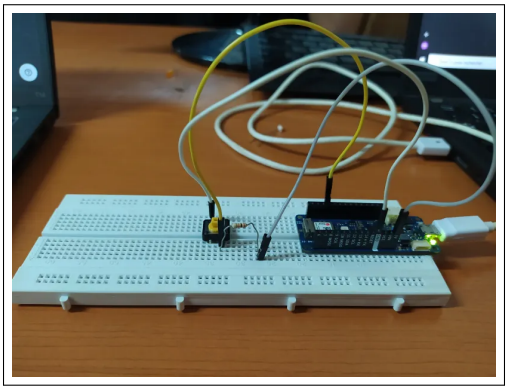

Figure 1.5: Simulation of the configuration.

3. De-bounce circuit

To eliminate signal noise caused by the mechanical bouncing of a pushbutton, a **debounce circuit** is added. A capacitor (typically 10 nF) is connected in parallel with the switch to smooth out rapid voltage changes.(see Figure 2)

With Debounce Capacitor: After adding the capacitor, the bouncing noise is filtered out, and the signal transitions cleanly. The console displays a single, stable transition on button press or release. This confirms that a debounce circuit is **necessary** for reliable input detection in real hardware systems.

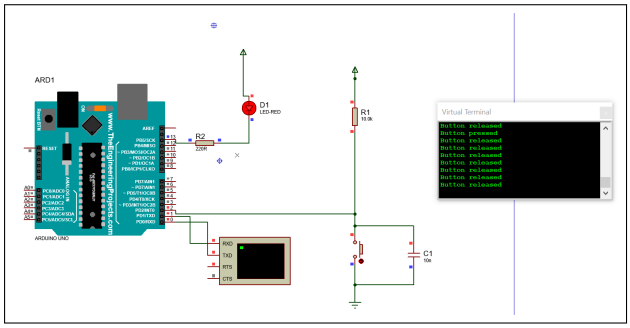

Figure 1.6: Debounce circuit on proteus

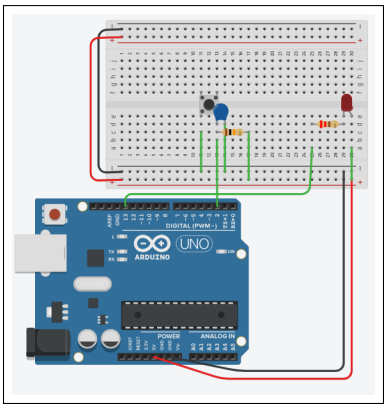

Figure 1.7: Debounce circuit on Tinker cad

4. LED Control on Light Threshold (ON/OFF)

4.1. Hardware

An LDR is connected to analog pin `A1` of the MKR1010 as part of a voltage divider to measure ambient light. A standard LED is connected to digital pin `A0` (configured as GPIO output on the MKR Wifi 1010) via a 220 Ω resistor.

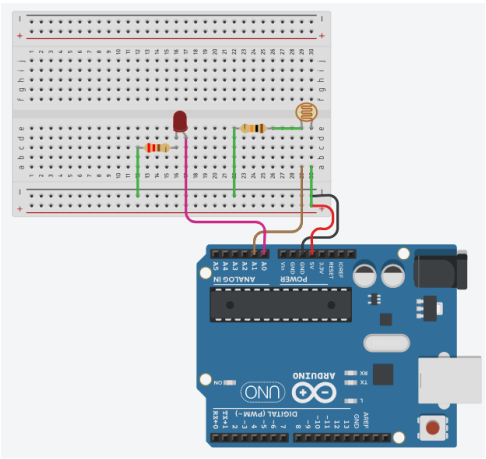

Figure 1.8: LED control on Light Threshold schematic.

4.2. Software

The program reads the analog light value using `analogRead()` and compares it to a threshold. The LED is controlled using `digitalWrite()` based on the measured light intensity.

```
const int LDR_PIN = A1;   // Analog input for LDR
const int LED_PIN = A0;   // Digital output for LED (preferd to reserve A0
    for analogue output)
int lightValue = 0;
```

```
int threshold = 500;      // Adjust this value based on ambient light

void setup() {
  Serial.begin(9600);          // Initialize serial communication
  pinMode(LDR_PIN, INPUT);     // Configure LDR pin as analog input
  pinMode(LED_PIN, OUTPUT);    // Configure LED pin as output
}

void loop() {
  lightValue = analogRead(LDR_PIN);  // Read light level by default 0-1023
   // to change to 12bits ADC analogReadResolution(12) in setup;

  Serial.print("Light intensity: ");
  Serial.println(lightValue);          // Output to serial monitor

  if (lightValue < threshold) {
    digitalWrite(LED_PIN, HIGH);       // dark alors Turn LED ON
  } else {
    digitalWrite(LED_PIN, LOW);        // bright alors Turn LED OFF
  }

  delay(500); // Delay for stability
}
```

4.3. Implementation

During execution, the system continuously reads the ambient light value. When the measured intensity is below the threshold (indicating low light), the LED is activated, simulating an automatic lighting system. The threshold value can be calibrated according to the lighting condition of the environment.

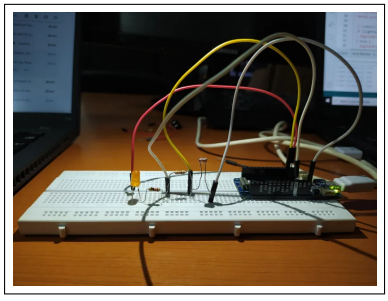

Figure 1.9: Simulation result: LED is OFF under bright ambient conditions (light value above threshold).

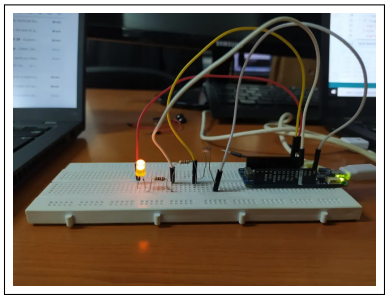

Figure 1.10: Simulation result: LED is ON under dark conditions (light value below threshold).

5. LED Light Dimming via Analog GPIO

5.1. Hardware

The LED is connected to the DAC-capable pin `A0` (configured as true analog output) through a 220 Ω resistor to limit the current. The LDR remains connected to analog input pin `A1` to sense ambient light (same schematic as Figure 1.8).

5.2. Software

The light value from the LDR is read using `analogRead()`. The value is then mapped to a DAC range (0–1023) using `map()` to control the brightness of the LED via `analogWrite()`. This creates a proportional light-dimming effect based on ambient light using a real analog voltage output .

```
const int LDR_PIN = A1;    // Analog input for LDR
const int LED_PIN = A0;    // DAC pin for LED
int lightValue = 0;        // ADC value from LDR (0-4095)
int dacValue = 0;          // DAC output value (0-1023)

void setup() {
  Serial.begin(9600);
  analogReadResolution(12);   // set ADC resolution to 12 bits (0-4095)
  analogWriteResolution(10);  // set DAC resolution to 10 bits (0-1023)
}

void loop() {
  lightValue = analogRead(LDR_PIN);              // read LDR value (0-4095)
  dacValue = map(lightValue, 0, 4095, 1023, 0); // inverse mapping to DAC
   output

  analogWrite(LED_PIN, dacValue);                // output analog voltage
  Serial.print("Light: ");
  Serial.print(lightValue);
  Serial.print(" | DAC Out: ");
  Serial.println(dacValue);

  delay(200); // Short delay for stability
}
```

Listing 1.3: LED brightness dimming control using an LDR and DAC output

5.3. Implementation

In this setup, the LED brightness changes smoothly with ambient light using true analog voltage output from the DAC. As the environment gets brighter, the LED dims (since the DAC value reduces). This approach avoids the visual flicker that can sometimes occur with PWM and is ideal for applications requiring smooth transitions and energy efficiency, such as smart lighting systems.

On the MKR 1010, the `analogRead()` function reads the input voltage with a resolution of up to 12 bits (0–4095). The result is inversely mapped to a 10-bit DAC output value (0–1023) using `map()`, which is then used to control the brightness of an LED via `analogWrite()` on pin `A0`.

6. Smart Streetlight Control Scheme

The smart streetlight concept leverages ambient light sensing and microcontroller-based decision-making to optimize public lighting. Each streetlight is equipped with an LDR to detect environmental brightness, and an LED lamp controlled by an analog output via a DAC-enabled GPIO line. During daylight, the LED remains off, conserving energy. At dusk or during low-light conditions, the LED automatically switches on or dims using an actual analog output voltage (via DAC) directly proportional to light intensity.

A centralized or distributed network can be used to gather data from multiple streetlights, enabling features like remote monitoring, fault detection, and scheduled dimming to reduce energy consumption during off-peak hours. Optional integration with motion sensors further enhances energy efficiency by activating lights only when pedestrians or vehicles are detected.

Figure 1.11: Smart streetlight .[1]

III. Conclusion

Through this lab, we explored the fundamental interaction between sensors, actuators, and microcontrollers. By combining digital and analog GPIO configurations, we were able to control devices such as pushbuttons, LEDs, and light sensors effectively. These basic concepts form the backbone of smart automation systems.

The LDR-based light sensing allowed real-time decision-making for energy-efficient lighting control, pushbutton interfacing introduced key concepts like pull-up/pull-down logic and debounce handling. furthermore , DAC-based dimming on the MKR1010

[1]Source: `https://tvilight.com/smart-street-lighting-for-residential-areas/`

provided true analog voltage control over LED brightness, demonstrating an even smoother transition of output power compared to traditional PWM methods. These elements came together in the implementation of a smart streetlight system a simple yet practical example of embedded intelligence at work.

In summery, this activiti demonstrated how Arduino platforms can serve as a solid foundation for IoT and **smart city applications**. It also highlighted the importance of writing clean software, designing effective hardware interfaces, understanding real world behavior beyond basic circuit theory.